

PRACTICE II

Pablo Fernández (100523036)

and

Marcos Villamiel (100522963)

INDEX

1. Introduction.....	3
2. Phase 1: State design and Reward function.....	4
2.1. State attributes selection.....	4
2.2. Reward function.....	5
3. Phase 2: Q-learning agent implementation.....	6
3.1. Implementation details.....	6
3.2. ϵ -Greedy implementation.....	8
4. Phase 3: Agent enhancement.....	10
4.1. Snake's body implementation.....	10
4.2. Training.....	10
4.3. Quantitative and qualitative evaluation.....	11
5. Conclusions.....	12

Introduction

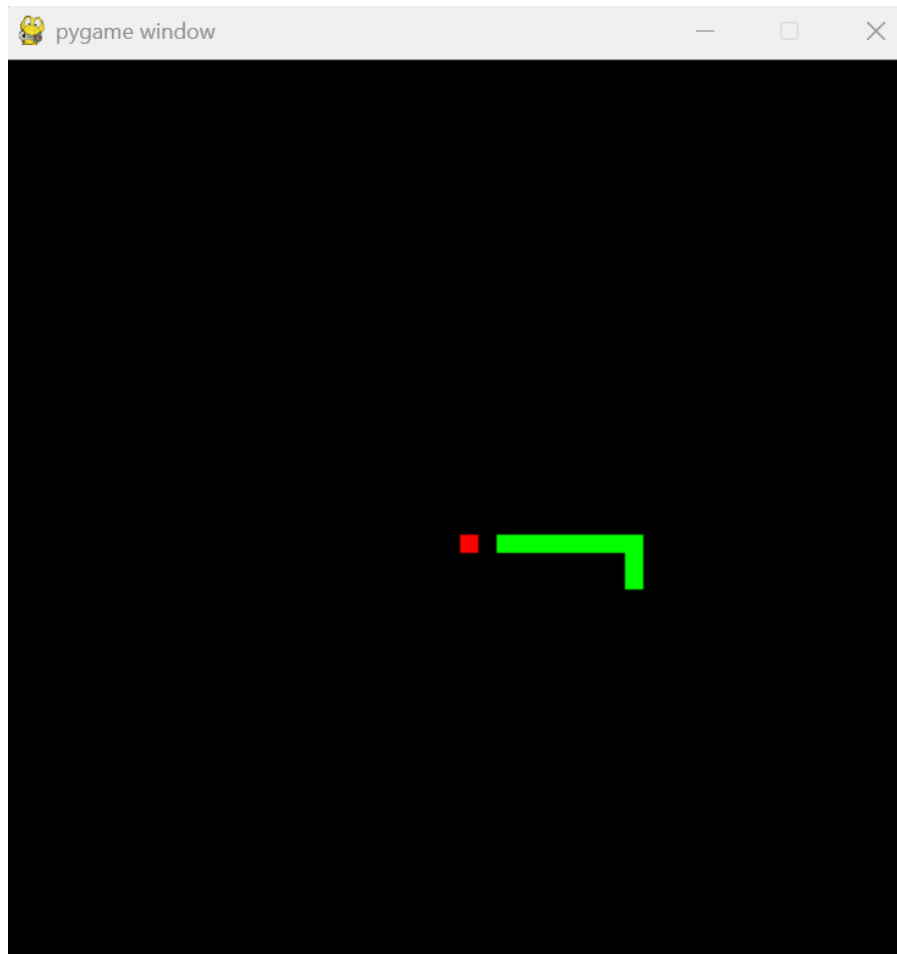
In this project, we use reinforcement learning to teach a Snake game agent how to play by itself. The goal is simple: the snake should learn to eat food and avoid collisions with its own body and the walls. To do this, we implement the Q-learning algorithm, a popular method in reinforcement learning.

The project is divided into three phases:

1. **Phase 1:** We design how the snake "sees" the game (state representation) and define rewards to guide its learning (e.g., +10 for eating food, -5 for crashing).
2. **Phase 2:** We build the Q-learning agent, train it on small boards, and adjust key settings like exploration rate (ϵ) and learning speed (α).
3. **Phase 3:** We improve the agent by making it aware of its own body, helping it survive longer in bigger and more complex boards.

We test the snake in different board sizes and measure its performance based on average score and survival time. The results show how well the agent learns and where it struggles.

This report explains our methods, experiments, and final results in a clear and simple way. By the end, you will understand how reinforcement learning can train an AI to play Snake.



Phase 1: State design and Reward function

State attributes selection

To start training our Q-learning agent we need to define the attributes we want for our state. We do that by implementing the code for the function `get_state(self)`. Q-learning works better for discrete or categorical values, so all the attributes that will be chosen are of this type. Since we want the snake to play in any board size, we will also select generic attributes, avoiding specific ones that could depend on the board size. Apart from that, we want nominal attributes with few values to keep the Q-table manageable. Finally, the selected attributes are the following:

- **food_left:** TRUE if the food is on the left of the snake's head, FALSE otherwise (boolean)
- **food_right:** TRUE if the food is on the right of the snake's head, FALSE otherwise (boolean)
- **food_up:** TRUE if the food is above the snake's head, FALSE otherwise (boolean)
- **food_down:** TRUE if the food is below the snake's head, FALSE otherwise (boolean)
- **danger_left:** TRUE if the position on the left of the snake's head is occupiable, FALSE otherwise (boolean)
- **danger_right:** TRUE if the position on the right of the snake's head is occupiable, FALSE otherwise (boolean)
- **danger_up:** TRUE if the position above the snake's head is occupiable, FALSE otherwise (boolean)
- **danger_down:** TRUE if the position below the snake's head is occupiable, FALSE otherwise (boolean)
- **dir_left:** TRUE if the direction of the snake is "LEFT", FALSE otherwise (boolean)
- **dir_right:** TRUE if the direction of the snake is "RIGHT", FALSE otherwise (boolean)
- **dir_up:** TRUE if the direction of the snake is "UP", FALSE otherwise (boolean)
- **dir_down:** TRUE if the direction of the snake is "DOWN", FALSE otherwise (boolean)

The chosen attributes provide a compact yet effective representation of the game state, perfectly suited for Q-learning. By focusing on discrete, boolean values, we ensure the state space remains manageable while capturing essential game dynamics. The attributes are generic (independent of board size) and minimalist (only 12 binary features). This design allows the Q-table to stay small enough for efficient learning while enabling the snake to make intelligent choices about movement and survival. The boolean approach also simplifies state identification, making the Q-learning updates straightforward and computationally efficient.

Reward function

The reward function is essential for the learning of the agent. It is like a “point system” that serves as a guide to the snake in order to learn how to play. The objective is to indicate which actions are good or bad for a certain situation by means of numerical values. This function is implemented as `calculate_reward(self)` and returns the reward (or penalty) depending on the behaviour of the agent in the game.

First of all, we will determine how our reward system works.

- If the snake dies, the penalty will be **-100**. We will apply a severe penalty to avoid the snake dying.
- If the snake eats the food, the reward will be **10**. This is to encourage the agent and prioritize eating the food.
- If the snake gets closer to the food, the reward will be **1**, but if it gets further it will be a penalty of **-1**. We will do this by calculating the distance between the snake and the food.
- If the snake does not progress, that is if it does none of the previously explained actions, there will be a little penalty of **-0.1**. This is to avoid unnecessary movements and little loops.

As we can observe, our rewards are distributed in a hierarchical way, and that is done to give more weight on more important aspects of the game. We prioritize surviving in the game, then getting closer to the food, after that progressing and finally eating the food.

As a summary:

Value	Situation	Objective
10	Eating the food	Maximize the score
1	Getting closer to the food	Learn to move towards the food
-0.1	Neutral move (no progression)	Avoid losing time (maximize the score)
-1	Getting further from the food	Discarding inefficient paths
-100	Dying	Prioritize not dying

These values are used to update the Q-table, teaching the snake which actions are the best in each state.

Phase 2: Q-learning agent implementation

In this phase, we will implement the Q-Learning algorithm for the Snake game by completing the missing functions (`calculate_reward`, `get_state`, `update_q_table`) and the main loop in `SnakeGame.py`. The goal is to design a state representation, update the Q-table using the Q-learning formula, and implement an ϵ -greedy action selection strategy. We will also tune hyperparameters (α , γ , ϵ) and ensure the Q-table is saved/loaded between games. The implementation should follow an incremental approach, starting with small boards and gradually increasing complexity while validating learning progress.

Implementation details

As explained in the previous phase, the functions `calculate_reward` and `get_state` are written as follows:

```
def calculate_reward(self):
    # Calculate and return the reward. Remember that you can provide
    # positive or negative reward.
    if self.check_game_over():
        return -100 # Big penalty for dying

    head_x, head_y = self.snake_pos
    food_x, food_y = self.food_pos

    if head_x == food_x and head_y == food_y:
        return 10 # Reward for eating food

    # Calculate Manhattan distance to food
    prev_distance = abs(self.snake_body[1][0] - food_x) + \
abs(self.snake_body[1][1] - food_y)
    current_distance = abs(head_x - food_x) + abs(head_y - food_y)

    if current_distance < prev_distance:
        return 1 # Reward for moving closer to food
    elif current_distance > prev_distance:
        return -1 # Penalty for moving away from food

    return -0.1 # Small penalty for no progress
```

We have decided to use the Manhattan distance to calculate the proximity to the food after each action taken in order to decide if returning a penalty (-1) or a reward (1).

```

def get_state(self):
    head_x, head_y = self.snake_pos
    food_x, food_y = self.food_pos

    food_left = food_x < head_x
    food_right = food_x > head_x
    food_up = food_y < head_y
    food_down = food_y > head_y

    danger_left = (head_x - 10 < 0) or ([head_x - 10, head_y] in
self.snake_body[1:])
    danger_right = (head_x + 10 >= self.frame_size_x) or ([head_x +
10, head_y] in self.snake_body[1:])
    danger_up = (head_y - 10 < 0) or ([head_x, head_y - 10] in
self.snake_body[1:])
    danger_down = (head_y + 10 >= self.frame_size_y) or ([head_x,
head_y + 10] in self.snake_body[1:])

    dir_left = self.direction == 'LEFT'
    dir_right = self.direction == 'RIGHT'
    dir_up = self.direction == 'UP'
    dir_down = self.direction == 'DOWN'

    state = (
        food_left, food_right, food_up, food_down,
        danger_left, danger_right, danger_up, danger_down,
        dir_left, dir_right, dir_up, dir_down
    )
    return state

```

This function returns a tuple with the selected attributes explained in the previous phase. Apart from these functions, the `update_q_table` function is computed as follows:

```

def update_q_table(self, state, action, reward, next_state,
terminal_state):

    if terminal_state:
        self.q_table[state, action] = (1 - self.alpha) *
self.q_table[state, action] + self.alpha * reward
    else:
        max_next_q = np.max(self.q_table[next_state])
        self.q_table[state, action] = ((1 - self.alpha) *
self.q_table[state, action] +
                                         self.alpha * (reward +
self.gamma * max_next_q))

```

The `update_q_table` function implements the core equation of the Q-Learning algorithm to update the Q-table values. If the state is terminal (game over), the Q-value is updated using only the immediate reward since there is no future state. Otherwise, it applies the Bellman equation: the new Q-value combines the current value (weighted by $1 - \alpha$) and the immediate reward plus the discounted maximum future Q-value (scaled by γ). The α parameter (learning rate) determines how much weight is given to new information versus prior knowledge. This update rule enables the agent to learn iteratively, balancing immediate and future rewards to optimize its action policy.

It takes as inputs current state, action, reward, next state and terminal state.

- **State:** The current state of the environment.
- **Action:** The action taken by the agent.
- **Reward:** The immediate reward received after taking the action.
- **Next state:** The new state the agent transitions into after performing the action.
- **Terminal state:** A boolean (True/False) indicating whether the game ended after this action.

ϵ -Greedy implementation

The ϵ -greedy strategy plays a crucial role in balancing exploration and exploitation during the learning process. This approach ensures that while our agent primarily follows its learned knowledge, it also explores new possibilities to discover potentially better strategies.

At the beginning of training, we set a relatively high ϵ value (0.7) to prioritize exploration. This means the agent has a 70% chance of selecting a random action and a 30% chance of choosing what it currently believes to be the best action based on its Q-table. As training progresses, we gradually decrease ϵ by multiplying it with a decay factor (0.995) after each episode. This allows the agent to shift its focus from exploration to exploitation as it accumulates more knowledge about the environment.

The implementation involves a simple but effective decision-making process. For each action selection, the agent generates a random number between 0 and 1. If this number is less than ϵ , the agent explores by choosing a random valid action. Otherwise, it exploits its current knowledge by selecting the action with the highest Q-value for the given state. This mechanism is particularly important in the early stages of learning, as it prevents the agent from getting stuck in suboptimal policies before it has adequately explored the state space.

To optimize performance, we implemented several adjustments. During evaluation phases, we set ϵ to 0 to assess the agent's pure exploitation capability. If the agent appears to be stuck in a local optimum during training, we temporarily increase ϵ to encourage more exploration. The decay rate was carefully chosen to ensure a smooth transition from exploration to exploitation,

allowing sufficient time for the agent to discover good strategies while eventually stabilizing its behavior.

This ϵ -greedy implementation proved particularly effective in our Snake game environment. The high initial ϵ value helped the agent discover fundamental strategies like moving toward food, while the gradual reduction allowed it to refine its approach and learn to avoid collisions. The balance between exploration and exploitation was critical in developing an agent that could both achieve high scores and adapt to different board configurations. Through experimentation, we found this approach provided the right compromise between discovering new strategies and capitalizing on learned ones, ultimately leading to a robust and competent Snake-playing agent.

Phase 3: Agent enhancement

1. Incorporating the Snake's Body into the State Representation

In this phase, the agent's state representation was improved to enhance decision-making by incorporating awareness of the snake's own body. The original state consisted of 12 binary features capturing relative food position, immediate danger in four directions, and current movement direction.

To incorporate the snake's body, we maintained a compact state space while including collision risk with the snake's own body. This was already partially handled through the `danger_left`, `danger_right`, `danger_up`, and `danger_down` flags in the environment (`get_state()` function in `snake_env.py`). These flags return `True` if the snake's head is about to collide with its own body or a wall. Therefore, the body-awareness was indirectly captured without explicitly encoding the entire body position, which would otherwise lead to an exponential increase in Q-table size.

This approach offers a good trade-off between capturing essential spatial information and managing the feasibility of Q-table size (which remains 2^{12} states).

2. Training on Multiple Board Configurations

To assess generalization, the agent was trained and evaluated on different board sizes: 150x300, 200x200, and 300x300 pixels. The training loop in `SnakeGame.py` supports configurable environment sizes via parameters passed to `SnakeGameEnv`.

Experiments were conducted over 100 episodes per configuration. The learning parameters were:

- Learning rate (α): 0.1
- Discount factor (γ): 0.9
- Initial exploration rate (ϵ): 0.1

The Q-table was persisted and reused across episodes to allow cumulative learning.

3. Quantitative and Qualitative Evaluation

Average rewards(across 100 episodes):

	150x300	200x200	300x300
Mean Reward	~100	~200	~250
Max Reward	251	520	1039

It is observable that there is more variance in the rewards of the chapters when the board size is smaller.

It is noticeable that the agent shows significant improvement as the episodes progress. Initially, its behavior appears random and often results in early termination due to collisions. However, after a number of episodes, the agent begins to exhibit more strategic movements—actively avoiding its own body and walls, and consistently navigating toward the food. This steady performance increase reflects the effectiveness of the Q-learning algorithm as the agent explores the environment and gradually learns from experience.

Final conclusions

This project explored how reinforcement learning, specifically Q-learning, can train an AI to play the Snake game. The results show that with the right design choices, the agent learns to navigate the game effectively, balancing survival and food collection.

1. State Design Matters

The 12 binary attributes provided enough information for the agent to learn without overwhelming the Q-table. Including collision checks with the snake's body was crucial for avoiding crashes.

2. Rewards Guide Learning

The reward system, with large penalties for dying (-100) and strong rewards for eating food (+10), helped the agent prioritize survival while still seeking food. Smaller rewards for moving closer to food refined its pathfinding.

3. Exploration vs. Exploitation

Starting with high exploration ($\epsilon = 0.7$) and gradually reducing it allowed the agent to discover strategies early and refine them later. This balance was key to avoiding poor long-term behavior.

4. Adaptability to Different Boards

The agent performed well across various board sizes, with better results in larger environments (e.g., 300x300). This suggests the learned strategies generalize effectively.

While successful, the approach has room for improvement. A more detailed state representation could help with complex scenarios, and techniques like Deep Q-Networks (DQN) could handle larger state spaces. Testing on more varied boards could further improve adaptability.

This project demonstrates how reinforcement learning can teach an AI to play Snake effectively. The methods used here could be applied to similar problems, showing how simple but well-designed systems can achieve strong results.